

BUILD A TRANSFORMER FROM SCRATCH

A complete beginner-to-builder guide: prerequisites, attention, encoder-decoder architecture, training, inference, tests and a realistic learning plan.

PYTORCH LEARNING EDITION

What you will build

A correct small encoder-decoder Transformer implemented with basic PyTorch layers. You will first make it learn a synthetic sequence-reversal task, then understand how to adapt it to translation. This guide deliberately avoids `nn.Transformer` until the end.

Recommended starting point

Comfortable Python, basic algebra, and beginner PyTorch. You do not need advanced calculus before starting. Learn the required mathematics when each idea appears.

ROADMAP

How to use this guide

Build one verified layer at a time. Do not copy the final architecture before you can explain the shapes inside attention.

The recommended order

Stage	You learn	You produce	Move on when...
0. Foundations	Python, tensors, linear layers, gradients	Small tensor exercises	You can predict shapes before running code
1. Attention	Q, K, V; scores; scaling; masks	Single-head attention	Output and attention shapes are correct
2. Multi-head	Split, transpose, combine heads	MultiHeadAttention	All heads recombine to <code>d_model</code>
3. Encoder	Residuals, norms, feed-forward, stacking	EncoderBlock + Encoder	Padding tokens cannot be attended to
4. Decoder	Causal attention and cross-attention	DecoderBlock + Decoder	Future tokens are hidden
5. Training	Teacher forcing, loss, optimizer	Tiny trained model	Loss falls and predictions improve
6. Inference	Autoregressive generation	Greedy decoder	Generation stops at EOS

Golden rule

For every function, write the expected input and output shape before writing its logic. Most Transformer bugs are shape, mask, device, or target-shifting bugs.

Suggested pace

- Fast but serious: 4 weeks, 60-90 minutes on weekdays and one longer Saturday session.
- Comfortable: 6 weeks, 45-60 minutes per day, with Sunday free.
- Do not measure progress by lines of code. Measure it by checkpoints you can explain without notes.

Scope

The original 2017 Transformer is an encoder-decoder model designed for sequence-to-sequence tasks. Modern decoder-only LLMs reuse many of the same ideas, but this guide first builds the complete original structure so self-attention, causal attention and cross-attention are all visible.

PART 1

Prerequisites

Everything you need before implementing the architecture, ordered by importance.

1. Python essentials

- Functions, classes, constructors, inheritance and calling an object as a function.
- Lists, dictionaries, list comprehensions, loops and unpacking.
- Imports, modules, virtual environments and reading tracebacks.
- Type hints are helpful but optional for the first version.

You should understand this pattern:

```
class Scale:
    def __init__(self, factor: float):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

double = Scale(2.0)
print(double(3)) # 6.0
```

2. Mathematics you actually need

Topic	Minimum understanding	Where it appears
Vectors and matrices	Shape, rows, columns, transpose	Embeddings and all linear operations
Matrix multiplication	Inner dimensions must match	$Q \times K^T$ and attention $Q \times V$
Dot product	A similarity score between two vectors	Query-key compatibility
Softmax	Turns scores into positive weights summing to 1	Attention probabilities and class probabilities
Square root	Scaling prevents very large logits	Divide scores by $\sqrt{d_k}$
Derivatives	Concept of gradients and chain rule	Backpropagation; PyTorch calculates it
Probability	Logits vs probabilities; cross-entropy	Vocabulary prediction loss

You do not need

Formal proofs, advanced linear algebra, or manually deriving every gradient before beginning. Those can deepen understanding later, but they should not block implementation.

3. PyTorch essentials

- Tensor creation, dtype and device: `torch.tensor()`, `.float()`, `.to(device)`.
- Shape tools: `.shape`, `reshape`, `view`, `transpose`, `permute`, `unsqueeze`.
- Operations: `matmul`, `softmax`, `masking` and `broadcasting`.
- Modules: `nn.Module`, `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`, `nn.ModuleList`.
- Training: `parameters`, `loss`, `loss.backward()`, `optimizer step`, `train()`, `eval()`, and `torch.no_grad()`.

PART 2

Set up the project

Use a small, reproducible environment and keep the first implementation easy to inspect.

Recommended environment with uv

```
mkdir transformer-from-scratch
cd transformer-from-scratch

uv init
uv add torch
uv run python -c "import torch; print(torch.__version__)"
```

Start with one file

Use one `main.py` while learning. Splitting the code too early makes it harder to follow the data flow. After the model passes tests, reorganize it:

```
transformer-from-scratch/
|-- pyproject.toml
|-- main.py           # first learning version
|-- transformer/
|   |-- attention.py
|   |-- positional.py
|   |-- encoder.py
|   |-- decoder.py
|   |-- model.py
|-- train.py
|-- infer.py
`-- tests/
    |-- test_attention.py
    |-- test_masks.py
```

Use small dimensions

Setting	Learning value	Reason
d_model	32 or 64	Fast, but large enough to split across heads
heads	4	Easy head_dim: $32 / 4 = 8$
layers	2	Enough to verify stacking
d_ff	4 x d_model	Matches the paper's broad pattern
dropout	0.1	Standard regularization; set 0 while debugging
max_len	32 or 64	Enough for synthetic sequences

Device rule

Use the device of the current tensor when creating temporary tensors. For example, create positions with `device=x.device`. This prevents a CPU/CUDA mismatch hidden inside the model.

Tensor shapes: your debugging language

A Transformer is easiest to understand as a controlled sequence of shape changes.

Standard symbols

Symbol	Meaning	Example
B	Batch size	2 sentences
S	Source sequence length	9 source tokens
T	Target sequence length	7 target tokens
D	Model or embedding dimension	64 features per token
H	Number of attention heads	4 heads
Dh	Dimension per head: D / H	16 features per head
V	Vocabulary size	10,000 token IDs

The central shape journey

```

Token IDs           [B, S]
-> embedding        [B, S, D]
-> split heads      [B, S, H, Dh]
-> transpose        [B, H, S, Dh]
-> Q @ K.transpose(-2, -1) [B, H, S, S]
-> softmax          [B, H, S, S]
-> attention @ V    [B, H, S, Dh]
-> transpose + combine [B, S, D]
```

A five-minute shape exercise

```

import torch

B, S, D, H = 2, 5, 32, 4
Dh = D // H
x = torch.randn(B, S, D)
split = x.reshape(B, S, H, Dh)
heads_first = split.transpose(1, 2)

print(x.shape)      # [2, 5, 32]
print(split.shape)  # [2, 5, 4, 8]
print(heads_first.shape) # [2, 4, 5, 8]
```

Checkpoint

Before continuing, explain why $D \% H == 0$ must be true and why transpose changes the interpretation of axes without changing the number of values.

PART 4

Embeddings and position

Convert token IDs to vectors and tell the model where every token occurs.

Token embedding

An embedding table has one learnable vector per token ID. It maps $[B, S]$ integer IDs to $[B, S, D]$ floating-point representations.

```
import torch
import torch.nn as nn

vocab_size, d_model = 100, 32
embedding = nn.Embedding(vocab_size, d_model)
token_ids = torch.tensor([[1, 5, 7], [4, 8, 2]])
x = embedding(token_ids)
assert x.shape == (2, 3, 32)
```

Sinusoidal positional encoding

The original paper adds fixed sine and cosine patterns to token embeddings. This gives every position a unique signal without learning a separate position table.

```
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 5000):
        super().__init__()
        position = torch.arange(max_len).unsqueeze(1).float()
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float()
            * (-math.log(10000.0) / d_model)
        )
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        # x: [B, S, D]
        return x + self.pe[:, :x.size(1)]
```

Why register_buffer?

The positional matrix is not trainable, but it must move with the model between CPU and GPU and be included in saved state. A registered buffer does exactly that.

Paper detail

The paper scales token embeddings by $\sqrt{d_model}$ before adding positional encoding. Include this in a faithful implementation.

Scaled dot-product attention

Build and test the primitive before introducing multiple heads.

The idea

Term	Question it represents
Query (Q)	What information is this token looking for?
Key (K)	What kind of information does this token contain?
Value (V)	What information should be returned if this token is relevant?

Core formula

$$\text{Attention}(Q, K, V) = \text{softmax}((Q \times K^T) / \sqrt{d_k}) \times V$$

Implementation

```
import math

def scaled_dot_product_attention(q, k, v, mask=None):
    # q: [B, H, Q, Dh]
    # k: [B, H, K, Dh]
    # v: [B, H, V, Dh]
    scores = q @ k.transpose(-2, -1) # [B, H, Q, K]
    scores = scores / math.sqrt(q.size(-1))

    if mask is not None:
        # mask must broadcast to [B, H, Q, K]
        scores = scores.masked_fill(~mask, float("-inf"))

    weights = torch.softmax(scores, dim=-1)
    output = weights @ v # [B, H, Q, Dh]
    return output, weights
```

Why each step exists

1. $Q \times K^T$ compares every query token with every key token.
2. Divide by $\sqrt{d_k}$ keeps dot products from becoming too large, which helps softmax and gradients remain useful.
3. Mask removes illegal locations before softmax by assigning an extremely negative score.
4. Softmax turns each query's scores into a distribution across key positions.
5. Weights $\times V$ creates a weighted summary of the useful value vectors.

Minimal test

```
B, H, S, Dh = 2, 4, 5, 8
q = torch.randn(B, H, S, Dh)
k = torch.randn(B, H, S, Dh)
v = torch.randn(B, H, S, Dh)
out, weights = scaled_dot_product_attention(q, k, v)

assert out.shape == (B, H, S, Dh)
assert weights.shape == (B, H, S, S)
assert torch.allclose(weights.sum(-1), torch.ones(B, H, S), atol=1e-5)
```

Multi-head attention

Project the input, let several heads attend in parallel, then recombine them.

Correct implementation

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x):
        # [B, L, D] -> [B, H, L, Dh]
        B, L, _ = x.shape
        return x.reshape(B, L, self.num_heads, self.head_dim).transpose(1, 2)

    def combine_heads(self, x):
        # [B, H, L, Dh] -> [B, L, D]
        B, H, L, Dh = x.shape
        return x.transpose(1, 2).contiguous().reshape(B, L, H * Dh)

    def forward(self, query, key, value, mask=None):
        q = self.split_heads(self.q_proj(query))
        k = self.split_heads(self.k_proj(key))
        v = self.split_heads(self.v_proj(value))

        scores = q @ k.transpose(-2, -1)
        scores = scores / math.sqrt(self.head_dim)
        if mask is not None:
            scores = scores.masked_fill(~mask, float("-inf"))

        weights = torch.softmax(scores, dim=-1)
        weights = self.dropout(weights)
        attended = weights @ v
        output = self.out_proj(self.combine_heads(attended))
        return output, weights
```

Self-attention vs cross-attention

Operation	Query	Key	Value	Score shape
Encoder self-attention	encoder x	encoder x	encoder x	[B, H, S, S]
Decoder self-attention	decoder x	decoder x	decoder x	[B, H, T, T]
Cross-attention	decoder x	encoder memory	encoder memory	[B, H, T, S]

Critical correction

Always apply the learned Q, K and V projections. Defining linear layers without calling them means the attention mechanism never learns those transformations. Also scale by $\sqrt{\text{head_dim}}$, not $\sqrt{\text{d_model}}$.

PART 7

Masks

Masks control which keys each query is allowed to use.

Source padding mask

Padding is not real content. A source mask hides PAD keys for every query.

```
def make_src_mask(src, pad_idx):
    # src: [B, S]
    # result: [B, 1, 1, S]
    return (src != pad_idx).unsqueeze(1).unsqueeze(2)
```

Target causal mask

During training, the complete target sentence is present, but position i must not see positions after i . The lower-triangular mask prevents that leak.

```
def make_causal_mask(length, device):
    # result: [1, 1, T, T]
    return torch.tril(
        torch.ones(length, length, dtype=torch.bool, device=device)
    ).unsqueeze(0).unsqueeze(0)

def make_trg_mask(trg, pad_idx):
    B, T = trg.shape
    pad_mask = (trg != pad_idx).unsqueeze(1).unsqueeze(2) # [B,1,1,T]
    causal = make_causal_mask(T, trg.device) # [1,1,T,T]
    return pad_mask & causal # [B,1,T,T]
```

Visual meaning

```
T = 4

1 0 0 0    query position 0 can use key 0
1 1 0 0    query position 1 can use keys 0..1
1 1 1 0    query position 2 can use keys 0..2
1 1 1 1    query position 3 can use keys 0..3
```

Mask tests

```
trg = torch.tensor([[1, 4, 5, 0]])
mask = make_trg_mask(trg, pad_idx=0)
assert mask.shape == (1, 1, 4, 4)
assert mask[0, 0, 1, 2].item() is False # future token hidden
assert mask[0, 0, 3, 3].item() is False # PAD key hidden
```

PART 8

Encoder

Combine self-attention, residual connections, normalization and a position-wise feed-forward network.

Feed-forward network

```
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
        )

    def forward(self, x):
        return self.net(x)
```

Encoder block: paper-style post-norm

```
class EncoderBlock(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, heads, dropout)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, src_mask):
        attn_out, _ = self.self_attn(x, x, x, src_mask)
        x = self.norm1(x + self.dropout(attn_out))
        ff_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ff_out))
        return x
```

Complete encoder

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, d_model, heads, layers,
                 d_ff, max_len, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position = PositionalEncoding(d_model, max_len)
        self.dropout = nn.Dropout(dropout)
        self.layers = nn.ModuleList([
            EncoderBlock(d_model, heads, d_ff, dropout)
            for _ in range(layers)
        ])

    def forward(self, src, src_mask):
        x = self.token_embedding(src) * math.sqrt(self.d_model)
        x = self.dropout(self.position(x))
        for layer in self.layers:
            x = layer(x, src_mask)
        return x # [B, S, D]
```

Checkpoint

With source shape [2, 9], the encoder should return [2, 9, D]. Sequence length and model dimension stay stable through every encoder block.

Decoder

Use masked self-attention, encoder-decoder cross-attention and a feed-forward network.

One correct decoder block

```
class DecoderBlock(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.1):
        super().__init__()
        self.masked_self_attn = MultiHeadAttention(d_model, heads, dropout)
        self.cross_attn = MultiHeadAttention(d_model, heads, dropout)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, memory, src_mask, trg_mask):
        masked, _ = self.masked_self_attn(x, x, x, trg_mask)
        x = self.norm1(x + self.dropout(masked))

        cross, _ = self.cross_attn(x, memory, memory, src_mask)
        x = self.norm2(x + self.dropout(cross))

        ff_out = self.ffn(x)
        x = self.norm3(x + self.dropout(ff_out))
        return x
```

Complete decoder

```
class Decoder(nn.Module):
    def __init__(self, vocab_size, d_model, heads, layers,
                 d_ff, max_len, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position = PositionalEncoding(d_model, max_len)
        self.dropout = nn.Dropout(dropout)
        self.layers = nn.ModuleList([
            DecoderBlock(d_model, heads, d_ff, dropout)
            for _ in range(layers)
        ])
        self.output = nn.Linear(d_model, vocab_size)

    def forward(self, trg, memory, src_mask, trg_mask):
        x = self.token_embedding(trg) * math.sqrt(self.d_model)
        x = self.dropout(self.position(x))
        for layer in self.layers:
            x = layer(x, memory, src_mask, trg_mask)
        return self.output(x) # [B, T, target_vocab_size]
```

Cross-attention shapes

Decoder queries have target length T ; encoder keys and values have source length S . The attention score matrix is therefore $[B, H, T, S]$. Code that assumes all three lengths are equal will fail when S and T differ.

Complete Transformer

Create masks, run the encoder once, and give its memory to every decoder layer.

Model wrapper

```
class Transformer(nn.Module):
    def __init__(self, src_vocab, trg_vocab, src_pad_idx, trg_pad_idx,
                  d_model=64, heads=4, layers=2, d_ff=256,
                  max_len=64, dropout=0.1):
        super().__init__()
        self.src_pad_idx = src_pad_idx
        self.trg_pad_idx = trg_pad_idx
        self.encoder = Encoder(
            src_vocab, d_model, heads, layers, d_ff, max_len, dropout
        )
        self.decoder = Decoder(
            trg_vocab, d_model, heads, layers, d_ff, max_len, dropout
        )

    def forward(self, src, trg_input):
        src_mask = make_src_mask(src, self.src_pad_idx)
        trg_mask = make_trg_mask(trg_input, self.trg_pad_idx)
        memory = self.encoder(src, src_mask)
        return self.decoder(trg_input, memory, src_mask, trg_mask)
```

Smoke test

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Transformer(
    src_vocab=20,
    trg_vocab=20,
    src_pad_idx=0,
    trg_pad_idx=0,
).to(device)

src = torch.tensor([[1, 5, 6, 2, 0], [1, 8, 7, 3, 2]], device=device)
trg = torch.tensor([[1, 6, 5, 2], [1, 3, 7, 2]], device=device)
logits = model(src, trg[:, :-1])

assert logits.shape == (2, 3, 20)
print(logits.shape) # torch.Size([2, 3, 20])
```

What the output means

For each example and every decoder input position, the model returns one unnormalized score (logit) for each target vocabulary token. Do not apply softmax before CrossEntropyLoss; it expects raw logits.

Training

Use teacher forcing, shifted targets, ignored padding and a tiny task that exposes bugs quickly.

Teacher forcing and target shifting

```
Full target:  [SOS, 9, 8, 7, EOS]
Decoder input: [SOS, 9, 8, 7]
Labels:       [9, 8, 7, EOS]
```

```
trg_input = trg[:, :-1]
labels = trg[:, 1:]
```

Training step

```
criterion = nn.CrossEntropyLoss(ignore_index=trg_pad_idx)
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)

model.train()
for src, trg in train_loader:
    src, trg = src.to(device), trg.to(device)
    trg_input = trg[:, :-1]
    labels = trg[:, 1:]

    optimizer.zero_grad()
    logits = model(src, trg_input)      # [B, T-1, V]
    loss = criterion(
        logits.reshape(-1, logits.size(-1)),
        labels.reshape(-1),
    )
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
```

First dataset: reverse a sequence

Train on generated sequences before using language. Example: source [SOS, 3, 6, 9, EOS] maps to target [SOS, 9, 6, 3, EOS]. This checks attention, masks, shifting, loss and generation without tokenization complexity.

Overfit one batch first

- Set dropout to 0.
- Reuse one tiny batch for hundreds of steps.
- The loss should become very small. If it cannot, do not add more data.
- Once it overfits, restore dropout and train on many generated batches.

Why this works

Overfitting a tiny batch is a diagnostic, not a final goal. It proves the model, loss, optimizer and data alignment can work together.

Inference

Generate one token at a time because the answer is not available during prediction.

Greedy decoding

```
@torch.no_grad()
def greedy_decode(model, src, sos_idx, eos_idx, max_len):
    model.eval()
    generated = torch.full(
        (src.size(0), 1), sos_idx,
        dtype=torch.long, device=src.device
    )

    for _ in range(max_len - 1):
        logits = model(src, generated)
        next_token = logits[:, -1].argmax(dim=-1, keepdim=True)
        generated = torch.cat([generated, next_token], dim=1)

        if torch.all(next_token.squeeze(1) == eos_idx):
            break

    return generated
```

Training vs inference

Training	Inference
Complete shifted target is available	Only source and generated prefix are available
All target positions processed in parallel	One new token is appended per step
Teacher forcing supplies correct previous tokens	Model consumes its own earlier predictions
Causal mask prevents future leakage	Future tokens do not exist yet

After greedy decoding

Implement beam search only after greedy decoding is correct. Beam search keeps several promising sequences, but it cannot repair a broken attention mask or a badly trained model.

Tests and debugging

Turn silent architectural mistakes into small, local failures.

Essential tests

Test	Expected result	Bug it catches
Attention output shape	[B, H, Q, Dh]	Bad transpose or reshape
Attention row sum	Approximately 1	Softmax on wrong axis
Causal mask	No position attends to future keys	Target leakage
Padding mask	PAD keys get zero probability	Model learning padding
Cross-attention	Score shape [B,H,T,S]	Assuming source and target lengths match
Backward pass	Every important parameter gets gradient	Unused projections/layers
Tiny batch overfit	Loss approaches zero	End-to-end wiring bug
Eval determinism	Same input gives same result	Dropout still active

Gradient check

```
loss.backward()
for name, parameter in model.named_parameters():
    if parameter.requires_grad and parameter.grad is None:
        print("NO GRADIENT:", name)
```

Debugging checklist

- Print shapes at the boundaries of attention, each block, encoder, decoder and final projection.
- Check bool masks and confirm that True means allowed in your implementation.
- Verify mask shapes broadcast to [B, H, Q, K].
- Confirm source and target token IDs are inside their vocabulary sizes.
- Confirm model and all input tensors are on the same device.
- Check that target input and label lengths match after shifting.
- Use dropout 0 and a fixed random seed while debugging.
- Watch for NaNs caused by rows where every key is masked.

Mistakes to avoid

Specific lessons from the reference repository and common first implementations.

Mistake	Why it is wrong	Correct approach
Define Q/K/V layers but never call them	The learned projections receive no gradients and attention uses raw inputs	Apply q_proj, k_proj and v_proj before splitting heads
Scale by $\sqrt{d_{\text{model}}}$	Each dot product uses head_dim features	Scale by $\sqrt{\text{head_dim}}$
Add an extra TransformerBlock in decoder	Duplicates attention/FFN and can mix source/target dimensions incorrectly	Exactly: masked self-attn, cross-attn, FFN
Pass decoder/encoder tensors in wrong Q/K/V order	Cross-attention must have target queries and source keys/values	<code>cross_attn(decoder_x, memory, memory)</code>
Default internal device to CUDA	Temporary masks may be created on a different device	Use <code>tensor.device</code> or registered buffers
Forget causal mask	Training can see future target tokens	Combine target padding and lower-triangular masks
Softmax before CrossEntropyLoss	CrossEntropyLoss already applies log-softmax	Pass raw logits
Train on translation immediately	Too many possible failure sources	Overfit one batch, then solve reversal, then real data

Faithful to the paper vs practical variation

Component	Original paper	Common modern variation
Position	Sinusoidal encoding	Learned positions or rotary embeddings
Normalization	Post-norm	Pre-norm for easier deep training
Activation	ReLU	GELU or SwiGLU
Optimizer schedule	Adam plus warmup/inverse sqrt	AdamW with varied schedules
Architecture	Encoder-decoder	Encoder-only or decoder-only

From synthetic data to translation

Add data complexity only after the model is mechanically correct.

Required data pipeline

1. Collect aligned source-target sentence pairs.
2. Train or select tokenizers for source and target languages.
3. Reserve PAD, SOS, EOS and unknown-token IDs.
4. Encode sentences, add SOS/EOS, pad within each batch and keep lengths reasonable.
5. Create train, validation and test splits without leakage.
6. Train while monitoring both training and validation loss.
7. Decode held-out examples and calculate a task metric such as BLEU in addition to qualitative review.

Practical expectations

A small CPU-trained model can demonstrate learning, but it will not reproduce the paper's translation quality. The goal of your first version is architectural understanding and correctness. High-quality translation requires much more data, compute, tokenizer work and training discipline.

Recommended progression

Level	Task	Success criterion
1	Overfit one fixed batch	Near-zero loss and exact output
2	Reverse random short sequences	Generalizes to unseen sequences
3	Copy or sort token sequences	Stable training with padding
4	Tiny language dataset	Reasonable outputs and falling validation loss
5	Translation dataset	Evaluation metric improves over baseline

A realistic 6-week plan

A structured learning path that fits around full-time development work.

Week	Main goal	Build	Checkpoint
1	Tensors, shapes, embeddings, matmul, softmax	Small shape notebook or script	Explain [B,S,D] and matrix multiplication
2	Single-head then multi-head attention	Tested MultiHeadAttention	Weights sum to 1; shapes pass
3	Masks, FFN, residuals, LayerNorm	EncoderBlock + Encoder	PAD tokens are ignored
4	Causal attention and cross-attention	DecoderBlock + full model	Different S and T work
5	Teacher forcing, loss and optimizer	Sequence reversal training	Overfits one batch and generalizes
6	Autoregressive inference and cleanup	Greedy decode, tests, README	EOS termination and reproducible demo

Weekly rhythm

- Monday: learn one concept and reproduce the smallest example.
- Tuesday-Wednesday: implement it without copying.
- Thursday: add shape assertions and tests.
- Friday: explain it aloud or in your README.
- Saturday: connect the week's pieces and debug end to end.
- Sunday: keep free, as requested in your normal planning routine.

Portfolio deliverable

Publish a clean repository with architecture explanation, tensor-shape table, tests, a small training curve, sample generations and a section called 'Mistakes I found and fixed.' That demonstrates understanding better than claiming the implementation is 'from scratch' without evidence.

FINAL

Master checklist

Use this before calling your Transformer implementation complete.

Done	Checkpoint
[]	I can explain B, S, T, D, H and head_dim.
[]	I understand why token embeddings require positional information.
[]	I can explain Q, K and V without memorizing a slogan.
[]	My attention divides by $\sqrt{\text{head_dim}}$.
[]	My Q, K and V projection layers are actually called and receive gradients.
[]	My masks are boolean and broadcast to [B, H, Q, K].
[]	My encoder hides padding positions.
[]	My decoder cannot see future target tokens.
[]	My cross-attention uses decoder queries and encoder keys/values.
[]	My model supports different source and target lengths.
[]	My final output shape is [batch, target_length, target_vocab].
[]	I shift target input and labels by one token.
[]	I pass raw logits to CrossEntropyLoss and ignore PAD labels.
[]	The model can overfit one tiny batch.
[]	The model learns the sequence-reversal task.
[]	Greedy inference begins with SOS and stops at EOS.
[]	CPU and GPU execution do not create device mismatches.
[]	I can explain each block and its input/output shapes from memory.

Best next action

Create the project, complete the tensor-shape exercise, then implement only `scaled_dot_product_attention`. Do not begin the encoder until its output and attention weights pass the tests in Part 5.

Primary references

- Vaswani et al., Attention Is All You Need: <https://arxiv.org/abs/1706.03762>
- PyTorch tensor documentation: <https://pytorch.org/docs/stable/tensors.html>
- PyTorch nn.Module documentation: <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
- The Annotated Transformer: <https://nlp.seas.harvard.edu/annotated-transformer/>

Remember: the goal is not to write the most code. The goal is to know exactly why each tensor exists, what shape it has, what it is allowed to attend to, and how the final loss teaches every projection.